

Modül 21: Kodunuzu Test Etmek

Bu müfredatın sıfır bağımlılık kuralı var, ve JUnit bir bağımlılık. Dolayısıyla bu modül testi çatısız anlatıyor, ki bu zaten daha kullanışlı ders olduğu ortaya çıkıyor: bir test çatısı sihir değildir, ve çalışan bir tanesinin tamamını görmek gerçek olanları anlamayı kolaylaştırır.

Ayrıca `assert` konusunu kapsıyor, çoğunlukla onun neden bir test aracı **olmadığını** bilmeniz için.

Neler Öğreneceksiniz

- Programınızdaki her `assert` ifadesinin neden varsayılan olarak hiçbir şey yapmadığı
- `assert` ifadesinin yanlış kullanıldığı iki yol, biri gerçek bir üretim hatası
- 39 satırda eksiksiz bir test çatısı
- Bir çatının, makul biçimde elle yazamayacağınız neyi kattığı

Başka Bir Dilden Geliyorsanız

	Python	Java
Yerleşik test koşucusu	<code>unittest</code> , <code>pytest</code>	JDK'da yok
Testlerde <code>assert</code>	normal yol	varsayılan olarak kapalı
Keşif	adlandırma geleneğiyle	anotasyonla, bir çatı üzerinden
Standart tercih	<code>pytest</code>	JUnit 5, fiilen evrensel

Önemli olan satır: Python'un `assert` ifadesi aksini istemedikçe açıktır. Java'nınki istemedikçe kapalıdır, ve neredeyse kimse istemez.

Ders

Assert'ler kapalı

`AssertionsAreOff.java` dosyasından:

```
-ea olmadan:  assertions enabled: false
-ea ile:      assertions enabled: true
```

Tespit, tanınmaya değer bir numara kullanıyor:

```
boolean enabled = false;
assert enabled = true;
```

Atama yalnızca assert'ler etkinse çalışır, çünkü aksi hâlde tüm ifade atlanır. Bir `assert` içindeki yan etkinin doğru olduğu çok az yerden biridir.

Buradan iki kural çıkar.

Bir assert içine asla mantık koymayın. Bu gerçek bir hatadır:

```
assert list.remove(item);
```

`-ea` ile testlerde çalışır ve üretimde sessizce hiçbir şey silmemeye başlar.

Çağıranlardan gelen argümanları doğrulamak için asla `assert` kullanmayın. Her zaman açık olan bir istisna kullanın:

```
if (n < 0) throw new IllegalArgumentException("n must not be negative");
```

`assert`, **yanlış olamayacağına inandığınız iç değişmezler** içindir: erişilemez bir switch default'u, kodunuzun az önce kurduğu bir durum. `UsingAssertions.java` ikisini de gösteriyor ve mesajlı biçimi kullanıyor, ki bu tek bir string'e mal olur ve mesajsız bir `AssertionError` hatasını kendini açıklayan birine çevirir.

Bu depo hakkında bir not: doğrulama scripti örnekleri düz `java <dosya>` ile çalıştırır, dolayısıyla assert'ler kapalıdır. `UsingAssertions.java` dosyası `// EXPECT: assertions` işaretleyicisi taşır, böylece script onu `-ea` ile çalıştırır. Bu olmadan `assert` öğreten bir örnek hiçbir şey yapmadan geçerdi.

39 satırda bir test çatısı

`MicroHarness.java` eksiksiz bir `Tests` sınıfı içerir. Gerçek çıktısı:

```
PASS    addition works
PASS    string concatenation
PASS    list size
FAIL    this one is wrong on purpose
        returned false
FAIL    this one throws
        IllegalStateException: boom
PASS    equality with a useful message
FAIL    and one that fails
        expected <5> but was <4>

7 run, 4 passed, 3 failed
```

Fark edilmeye değer üç tasarım kararı, çünkü her gerçek çatı aynılarını verir:

1. **Fırlatan bir test, çalıştırmanın çökmesi değil bir başarısızlıktır.** Çatı `Throwable` yakalar, ki bu bunun doğru olduğu az sayıda yerden biridir: bir testin ona yapabileceği her şeyden sağ çıkmak zorundadır.

2. **Başarısızlıklar bir ayrıntı taşır**, dolayısıyla çıplak bir `FAIL` yerine `expected <5> but was <4>` görünür.
3. **Gerçek bir çatı, bir şey başarısız olduğunda sıfır dışı çıkar**, böylece CI fark eder. Bu çatı çıkmıyor, çünkü bu deponun kendi doğrulamasından temiz geçmek zorunda.

Bir çatının neler kattığı

`WhatJUnitAdds.java` dosyasından:

1. Keşif. Mikro çatı her testin elle kaydedilmesini gerektirir. Bir satır unutun ve test sessizce var olmaz, ve hiçbir şey hiç kaydedilmemiş bir testi bildirmez. JUnit `@Test` anotasyonunu tarar ve o başarısızlık biçimi kaybolur.

2. Yalıtım. Her `@Test` yeni bir örnek alır, dolayısıyla durum testler arasında sızamaz. Onsuz:

```
test one added an item; list size is now 1
test two starts with size 1, not 0
```

Yalnızca önceki çalıştığı için geçen bir test, hiç test olmamasından daha kötüdür, çünkü biri sıralamayı değiştirdiğinde başarısız olur.

3. Assert mesajları. On alanlı bir nesnede farklı alanı gösteren `assertEquals`, hata ayıklama oturumunun tamamıdır.

4. Elle yazmaya yer olmayan şeyler: parametrelili testler, `assertThrows`, zaman aşımaları, etiketleme, IDE entegrasyonu ve CI'nın okuyabileceği standart bir rapor biçimi.

```
class CalculatorTest {
    @Test
    void addsTwoNumbers() {
        assertEquals(4, Calculator.add(2, 2));
    }

    @Test
    void rejectsNegativeInput() {
        assertThrows(IllegalArgumentException.class, () -> Calculator.sqrt(-1));
    }
}
```

Gerçek bir projede ilk gün JUnit 5 ekleyin. Her yerde varsayılan ve onunla ilgili tartışmalı hiçbir şey yok. Bu modülden taşınan şey çatı değil: bir testin bir şeyi denetleyip bildiren bir metod olduğunu, testler arası yalıtımın bilinçli bir özellik olduğunu ve `assert` ifadesinin bir test aracı olmadığını bilmek.

Çalıştırın

```
java modules/21-testing-your-code/examples/AssertionsAreOff.java
java -ea modules/21-testing-your-code/examples/AssertionsAreOff.java

java -ea modules/21-testing-your-code/examples/UsingAssertions.java
java modules/21-testing-your-code/examples/MicroHarness.java
java modules/21-testing-your-code/examples/WhatJUnitAdds.java

./scripts/verify-examples.sh modules/21-testing-your-code
```

Birincisini her iki biçimde de çalıştırın. Fark, dersin kendisi.

Sık Yapılan Hatalar

`assert` ifadesinin çalıştığını varsaymak. Biri `-ea` geçirmedikçe çalışmaz.

Bir `assert` içinde yan etkiler. `assert list.remove(x)` üretimde çalışmayı durdurur.

Çağırının argümanlarını `assert` ile doğrulamak. Bir istisna kullanın.

Birbirine bağımlı testler. İkincisi birincisi çalıştığı için geçer, ta ki biri sıralamayı değiştirene kadar.

Başarısızlıkta ayrıntısı olmayan bir test. "FAIL" size hiçbir şey söylemez; beklenen ve gerçekleşen değerler her şeyi söyler.

İlk istisnada ölen bir çatı. Tek bir kötü test çalıştırmanın tamamını düşürmemeli.

Gerçek bir proje için kendi çatınızı yazmak. Yukarıdaki her şey JUnit kullanmanın lehine argümandır, aleyhine değil.

Özet Çıkarımlar

- **Assert'ler varsayılan olarak kapalıdır**, dolayısıyla `assert` bir test mekanizması değildir.
- Bir `assert` içine asla mantık ya da argüman doğrulaması koymayın. Yalnızca iç değişmezler.
- Bir test çatısı, bir şeyi denetleyip bildiren bir metottur. Otuz dokuz satır bunu kanıtlamaya yeter.
- Bir çatıda `Throwable` yakalayın ki tek bir kötü test çalıştırmayı bitirmesin.
- Başarısızlıklar beklenen ve gerçekleşeni taşımalı, yoksa hata ayıklama hiçbir yerden başlar.
- Gerçek projelerde JUnit kullanın. Bu modül onun neden var olduğudur, yerine geçeni değil.

Ödev

[homework/README.md](#)

Çatıyı bir framework'ün size vereceği özelliklerle genişletin, sonra `assert` ifadesinin gizlediği hatayı bulun. Referans çözüm solutions/21-testing-your-code/ altında.